Day 16: Exception Handling

You have already met errors many times: ValueError from int("abc"), FileNotFoundError from a missing file, KeyError from a bad dictionary key. Until now they crashed your programs. Today you learn to **catch** them, so a tool can respond sensibly instead of dying. (You saw a glimpse of try/except on Days 13 to 15; now you get the full picture.)

1. Understanding Runtime Errors

Python errors fall into three groups.

Type	When it happens	Example
Syntax error	Before the program runs; the code is malformed	print("hi"
Runtime error (exception)	While running; valid code hits a problem	10 / 0
Logic error	Program runs but gives wrong results	Using + instead of -

Only runtime errors can be caught. Syntax errors must be fixed, and logic errors are found by testing.

What an exception is

When something goes wrong, Python **raises an exception**, an object describing the problem. If nothing handles it, the program stops and prints a **traceback**:

python

```
numbers = [1, 2, 3]

print(numbers[5])
```

```
Traceback (most recent call last):
  File "demo.py", line 2, in <module>
    print(numbers[5])
IndexError: list index out of range
```

How to read a traceback: start at the **bottom**. The last line gives the **error type** and **message**; the lines above show the **file** and **line number** where it happened. In nested function calls, the last File line is closest to the cause.

Common exceptions

Exception	Typical cause
ValueError	Right type, bad value: int("abc")
TypeError	Wrong type: "5" + 3
ZeroDivisionError	10 / 0
IndexError	List index out of range
KeyError	Missing dictionary key

FileNotFoundError	File does not exist
PermissionError	Not allowed to read or write the file
FileExistsError	"x" mode on an existing file
NameError	Variable not defined
AttributeError	Method doesn't exist: 5.append(1)
KeyboardInterrupt	User pressed Ctrl + C

Exceptions form a hierarchy. For example, FileNotFoundError and PermissionError are both kinds of OSError, and KeyError and IndexError are both kinds of LookupError. Catching a parent type catches all its children.

Why it matters in security

- A scanner that crashes on one unreachable host never finishes the others.
- A log parser that dies on one damaged line misses everything after it.
- Unhandled errors can **leak information**: tracebacks reveal file paths and code structure. In real applications, users should see a short, safe message while details go to a log.

2. try and except

Basic structure

python

```
try:
    number = int(input("Enter a number: "))
    print(f"Double is {number * 2}")
except ValueError:
    print("That was not a valid number.")
```

How it works:

1. Python runs the try block line by line.
2. If **no error** occurs, the except block is skipped.
3. If an error occurs, Python **jumps immediately** to the matching except; the rest of the try block is skipped.
4. The program carries on after the whole structure.

python

```
try:
    print("Step 1")
    x = 10 / 0
    print("Step 2")          # never runs
```

```
except ZeroDivisionError:
    print("Cannot divide by zero")
print("Program continues")
```

Output:

```
Step 1
Cannot divide by zero
Program continues
```

Catching several exception types

Separate blocks for different responses:

python

```
try:
    a = int(input("a: "))
    b = int(input("b: "))
    print(a / b)
except ValueError:
    print("Please enter whole numbers.")
except ZeroDivisionError:
    print("b cannot be zero.")
```

One block for several types (use a tuple):

python

```
except (ValueError, TypeError):
    print("Bad input")
```

Getting the error message

python

```
try:
    int("abc")
except ValueError as e:
    print(f"Error details: {e}")
```

Output: Error details: invalid literal for int() with base 10: 'abc'

else and finally

python

```
try:
    number = int(text)
except ValueError:
    print("Invalid number")
else:
    print(f"Got {number}")          # runs only if NO error occurred
finally:
    print("Done checking")          # ALWAYS runs
```

- else holds code that should run only on success. It keeps the try block small.
- finally always runs, whether or not there was an error. It is used for cleanup. (with open(...) already handles file closing, so you will use finally less often.)

Order of the full structure

try → except (one or more) → else → finally

Raising your own exceptions

You can trigger an exception deliberately with raise:

python

```
def set_port(port):
    if not 0 <= port <= 65535:
        raise ValueError(f"Port out of range: {port}")
    return port


try:
    set_port(70000)
except ValueError as e:
    print(e)                        # Port out of range: 70000
```

Raising is how a function says "I cannot do this with what you gave me," and the caller decides what to do.

Good habits

1. Catch specific exceptions. Avoid a bare except:.

python

```
# BAD: hides every problem, even typos and Ctrl+C
try:
```

```
    do_something()
except:
    pass


# GOOD
try:
    do_something()
except ValueError:
    print("Bad value")
```

A bare except: or except Exception: followed by pass **silently swallows bugs,** which is dangerous in security tools because failures go unnoticed.

2. Keep try blocks small. Wrap only the line that can fail.

3. Don't use exceptions to hide bad logic. Prevent predictable problems with if checks where that's simpler.

4. Handle errors where you can do something useful, and let others pass up to a caller who can.

5. Report, don't hide. Print a clear message or write to a log (Day 15's log_event).

3. Handling Invalid Input

On Day 7 you validated input with isdigit(), which can't handle negatives or decimals. try/except is more reliable: **just try the conversion and catch the failure.**

A reusable integer reader

python

```
def ask_int(prompt):
    """Keep asking until the user enters a whole number."""
    while True:
        text = input(prompt).strip()
        try:
            return int(text)
        except ValueError:
            print("Please enter a whole number.")
```

It handles "42", "-5", " 7 ", and rejects "abc", "3.5", and "".

With a range

python

```
def ask_int_in_range(prompt, low, high):
```

```
    while True:
        try:
            value = int(input(prompt).strip())
        except ValueError:
            print("That is not a whole number.")
            continue

        if low <= value <= high:
            return value
        print(f"Please enter a number from {low} to {high}.")
```

python

```
port = ask_int_in_range("Port (0-65535): ", 0, 65535)
```

A reusable float reader

python

```
def ask_float(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Please enter a number.")
```

Handling Ctrl + C and end-of-input

python

```
try:
    name = input("Name: ")
except KeyboardInterrupt:
    print("\nCancelled by user.")
except EOFError:
    print("\nNo input available.")
```

EOFError happens when input ends unexpectedly, such as piped input running out.

Validating data from other places

Exceptions help any time data comes from outside your program:

python

```
def parse_line(line):
    """Return (user, count) or None for a bad line."""
    try:
        user, count_text = line.split(",")
        return user.strip(), int(count_text)
    except ValueError:                # wrong number of fields OR bad number
        return None
```

Here ValueError covers both a wrong field count (unpacking) and a non-numeric count. Inside a loop, this lets you skip damaged lines and continue.

Handling dictionary lookups

python

```
config = {"port": 22}


try:
    timeout = config["timeout"]
except KeyError:
    timeout = 5                   # fallback default
```

config.get("timeout", 5) does the same in one line. Use whichever is clearer.

Security note: validate, don't trust

Catching ValueError handles *bad format*, but not *bad intent*. A valid integer can still be out of range, and a valid string can still be malicious. Always combine exception handling with range, length, and allowed-value checks.

4. Handling Missing Files

Basic handling

python

```
try:
    with open("auth_sample.log", "r", encoding="utf-8") as f:
        content = f.read()
except FileNotFoundError:
    print("Error: the log file was not found.")
else:
    print(f"Read {len(content)} characters")
```

Other file problems

python

```
try:
    with open(filename, "r", encoding="utf-8") as f:
        content = f.read()
except FileNotFoundError:
    print(f"'{filename}' does not exist.")
except PermissionError:
    print(f"You don't have permission to read '{filename}'.")
except IsADirectoryError:
    print(f"'{filename}' is a folder, not a file.")
except UnicodeDecodeError:
    print(f"'{filename}' is not a valid UTF-8 text file.")
```

To catch **any** file-system problem in one block, use the parent class:

python

```
except OSError as e:
    print(f"Could not read the file: {e}")
```

Put specific handlers **before** general ones, because Python uses the first match.

Asking until the file exists

python

```
def ask_filename():
    while True:
        name = input("File name (or 'quit'): ").strip()
        if name.lower() == "quit":
            return None
        try:
            with open(name, "r", encoding="utf-8") as f:
                return f.read()
        except FileNotFoundError:
            print("File not found. Try again.")
        except OSError as e:
```

```
        print(f"Cannot read file: {e}")
```

Using it in a function (return a status)

python

```
def read_lines(filename):
    """Return a list of lines, or None if the file can't be read."""
    try:
        with open(filename, "r", encoding="utf-8") as f:
            return f.read().splitlines()
    except FileNotFoundError:
        print(f"Error: '{filename}' not found.")
    except PermissionError:
        print(f"Error: no permission to read '{filename}'.")
    return None
```

The caller checks if lines is None: and decides what to do.

Missing files vs. checking first

python

```
import os
if os.path.exists(name):
    open(name)          # file could still vanish between the check and the open
```

The safer approach is **"try it and handle the failure"** rather than "check, then act," because the situation can change between the two steps. This race condition is a real class of security bug.

Writing safely

python

```
try:
    with open("report.txt", "x", encoding="utf-8") as f:
        f.write("data\n")
except FileExistsError:
    print("report.txt already exists. Not overwriting.")
except PermissionError:
    print("Cannot write here: permission denied.")
```